Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій
Кафедра інформаційних технологій

ЗВІТ

з лабораторної роботи № 6

з дисципліни «Основи програмування»

«Цикли»

Виконала:
студентка групи ФЕП-13
Савчак Дарина
Перевірив:
асистент Васюта В. М.

Львів 2026

Мета роботи:

Вивчити поняття і застосування циклів.

Обладнання та програмне забезпечення:

– IBM сумісна персональна обчислювальна машина;

– онлайн компілятор мови програмування Сі, доступний за посиланням

https://repl.it/languages/c

Завдання до роботи:

Написати програму на мові програмування Сі, котра реалізує наступні функції:

– використання оператора for;

– використання оператора while;

– використання оператора do/while;

– використання оператора break;

– використання вкладених операторів continue;

Теоретичні відомості:

В мові програмування (Сі) цикли потрібні для того, щоб повторювати одну й ту саму дію або шматок коду декілька разів, поки виконується певна умова.

Основні види циклів та оператори для роботи з ними:

1. Цикл **for**
   Найчастіше використовується тоді, коли ми наперед знаємо, скільки саме разів потрібно повторити код. Спочатку задається початкове значення змінної, потім перевіряється умова, і після кожного проходу значення змінної змінюється (модифікується).Умову цикл перевіряє ще до того, як виконати код. Якщо умова від початку хибна, код усередині не виконається жодного разу. Також у цьому циклі можна залишати поля порожніми (наприклад, для створення нескінченного циклу або затримки).
2. Цикл **while**

(цикл із передумовою) Застосовується тоді, коли кількість повторень наперед невідома, а все залежить від певної умови. Спочатку перевіряється умова. Якщо вона правильна (істинна), виконується код усередині циклу. Це повторюється до тих пір, поки умова не стане хибною.

3. Цикл **do/while**

(цикл із післяумовою) Схожий на while, але з однією важливою відмінністю: Спочатку виконується сам код, і тільки в кінці перевіряється умова. Код усередині цього циклу гарантовано виконається хоча б один раз, навіть якщо умова з самого початку не виконується. Зручно використовувати, наприклад, для перевірки зчитування даних від користувача.

Оператори керування циклами

break — використовується для того, щоб перервати та повністю закрити цикл, не чекаючи, поки він добіжить кінця сам.

continue — дозволяє пропустити всі наступні команди в поточному проході (ітерації) та одразу перейти до наступного повторення циклу.

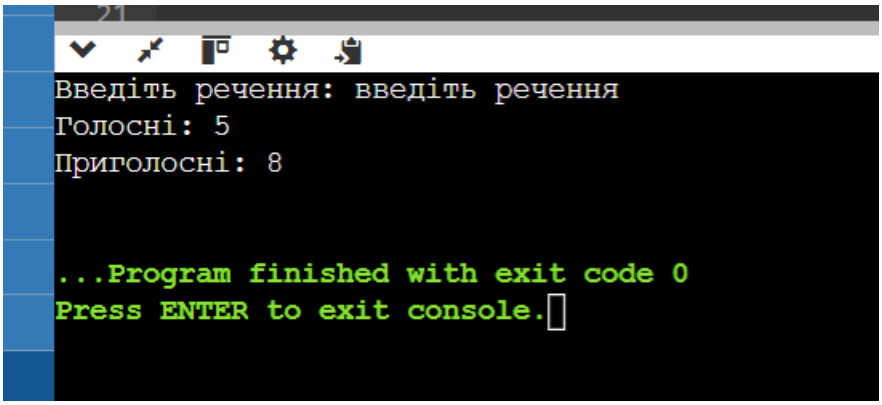
ХІД РОБОТИ

Завдання 1. Підрахунок голосних і приголосних у реченні (цикл for) Програма зчитує рядок (підтримка кирилиці через wchar_t і локаль UTF-8), проходить символи циклом for, переводить у нижній регістр і перевіряє належність до списків голосних/приголосних через wcschr.



Зображення 1. підрахунок голосних і приголосних

Результат виконання:



Коли ми вводимо фразу «введіть речення», програма видає правильний результат: 5 голосних і 8 приголосних. А спеціальні команди (wchar_t) потрібні обов’язково, щоб програма не «губила» і не пропускала українські літери.

Завдання 2. Сума перших 100 чисел без номера в журналі та 33 (while + continue)

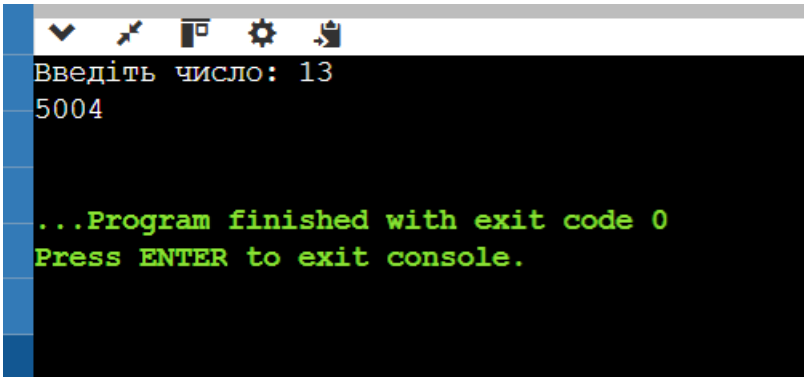
У циклі while від 1 до 100 накопичується сума. Числа 33 і введене

користувачем (порядковий номер у журналі) пропускаються оператором continue.



Зображення 2. while, continue

Результат виконання:



Зображення 3. номер 13 - сума 5004

Сума чисел від 1 до 100 дорівнює 5050. Виключено 33 і 13: 5050 − 33 − 13 = 5004 — результат збігається з виводом програми.

Завдання 3. N чисел: min, max, sum, avg (цикл for)

Користувач обирає кількість чисел N (від 7 до 12). За допомогою циклу **for** вводяться ці числа, після чого програма знаходить мінімальне, максимальне, загальну суму та середнє арифметичне значення. Змінні для мінімуму й максимуму спочатку отримують

значення першого введеного числа, щоб уникнути помилок у пам’яті.

main.c

```c
#include <stdio.h>
int main() {
    int n;
    printf("Введіть число від 7 до 12: ");
    scanf("%d", &n);

if (n > 12 || n < 7) {
    printf("Введіть коректне число\n");
    return 1;
}

double array[n];
double min;
double max;
double sum = 0;

for (int i = 0; i < n; i++) {
    printf("Введіть число: ");
    scanf("%lf", &array[i]);

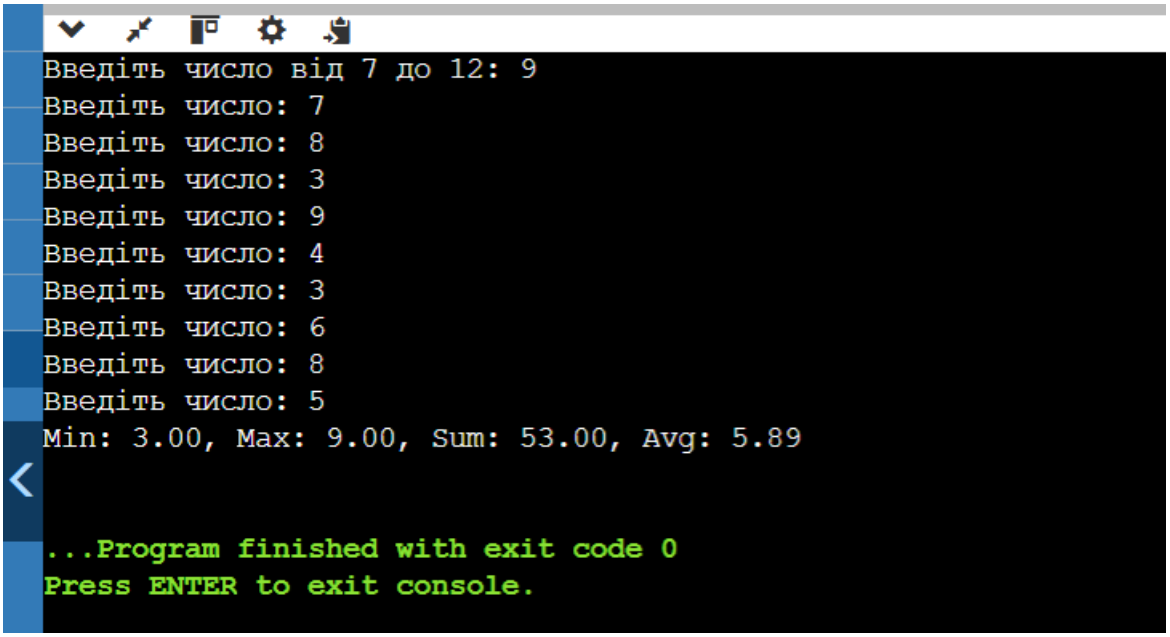
    if (i == 0) {
        min = array[i];
        max = array[i];
    } else {
        if (array[i] < min) {
            min = array[i];
        }
        if (array[i] > max) {
            max = array[i];
        }
    }
    sum += array[i];
}
```

```c
double avg = sum / n;

printf("Min: %.2lf, Max: %.2lf, Sum: %.2lf, Avg: %.2lf\n", min, max, sum, avg);

return 0;
}
```

Результат виконання:



Зображення 4. N = 9 – Min = 3, Max = 9, Sum = 53, Avg = 5.89

Для набору 7, 8, 3, 9, 4, 3, 6, 8, 5: min = 3, max = 9, sum = 53, avg = 5.89. Перевірка діапазону N та ініціалізація min/max першим елементом забезпечують коректну роботу

ВИСНОВОК:

Під час виконання лабораторної роботи було створено три програми та на практиці розібрано основні нюанси роботи з циклами в мові С.

У першому завданні вдалося перевірити, що для роботи з українськими літерами в Linux обов'язково треба налаштовувати локаль і використовувати широкі символи **wchar_t** разом із правильним **специфікатором %** , інакше програма просто ігнорує введення тексту. У другому завданні з циклом **while** вдалося звернути увагу на правильні умови завершення циклу, щоб жодне число не випадало з підрахунку — було успішно виключено число 33 та мій порядковий номер 13. У третьому завданні вдалося розібратися з пошуком найменшого і найбільшого значень. Щоб програма не брала випадкові дані з пам'яті, її було налаштовано так, щоб min і max брали значення першого введеного числа.

Для набору з N = 9 чисел (7, 8, 3, 9, 4, 3, 6, 8, 5) програма видала такі результати: min = 3, max = 9, sum = 53, avg= 5.89. Перевірка діапазону для N та правильна ініціалізація змінних забезпечили чітку роботу програми.

Усі програми повністю налаштовані, працюють без помилок і дають правильні результати.